PACTZERO

2x2 UPA Antenna Controller

Technical Specifications

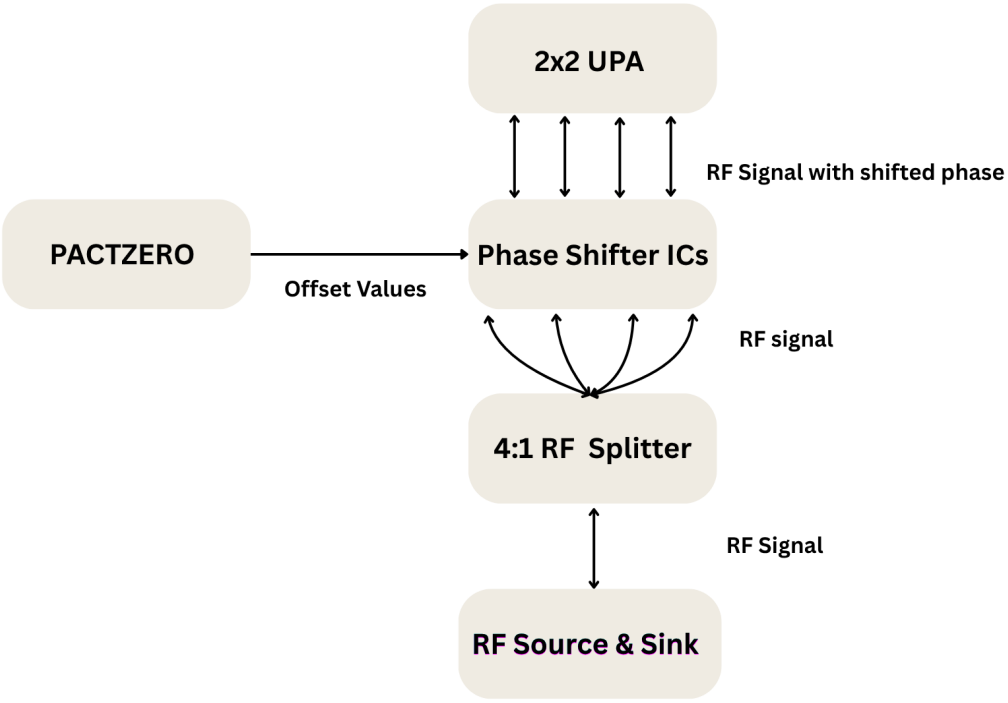
Index

1. Project Overview

A. Introduction

P.A.C.T. stands for Phased Array Controller and Transceiver. It is a hardware-software co-design to be used with Zync7000 SOC to control a UPA module for target azimuth and elevation angle and transceive radio signals.

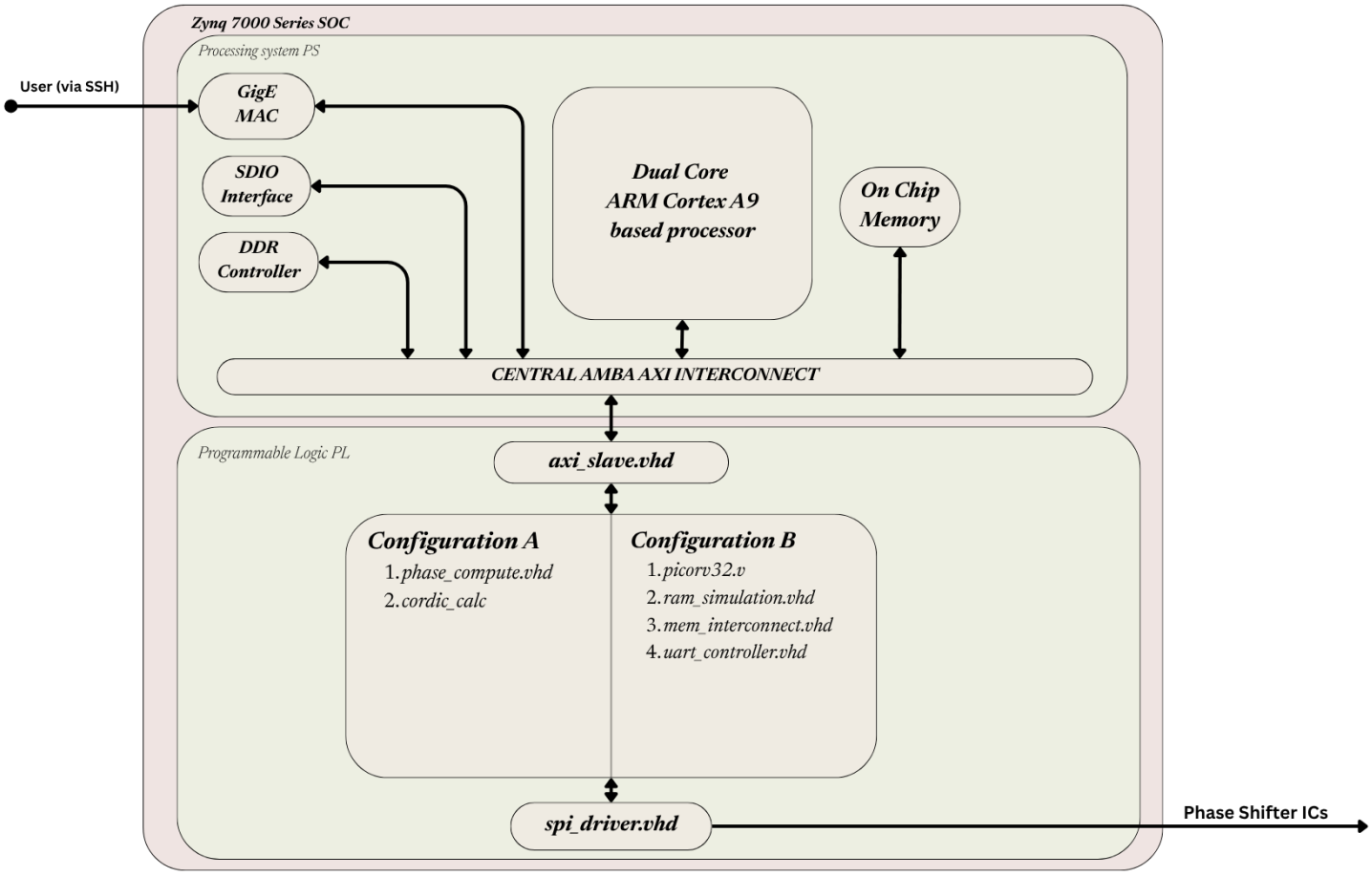
PACT ZERO is the zeroth iteration or prototype version in the PACT series and its functionality only includes controlling a 2x2 UPA .



2. System Architecture

A. Zynq-7000 XC7Z020 SoC

The Zynq-7000 series SOC is a single chip containing two distinct halves The Processing System (abbreviated as PS), is a fixed silicon specifically ARM CORTEX A9. Runs on PetaLinux, hosts kernel driver and userspace applications accessed by GigE MAC interface.



An external DDR3 SDRAM acts as the PS working memory. FSBL loads U-Boot and FIT image contents into DDR3 during boot. At runtime DDR3 holds the Linux kernel, rootfs, loadable kernel modules, and application binaries. [Read more about files in DDR3 in Section 2.B.]

Also the system requires an external SD to store boot image and the FIT image. [Read more about files in SD card in Section 2.C.]

The Programmable Logic (abbreviated as PL) is the FPGA fabric. Used for implementing the design described by RTL source files.

B. External DDR3 SDRAM

MANAGED BY ZYNQ PS MEMORY CONTROLLER

An External DDR3 Memory is required for storing

A. During boot time
   1. U-Boot, loaded from SD card by FSBL
   2. FIT image (raw) , loaded from SD card by U-Boot in DDR3
      a. Linux kernel (compressed)
      b. Device tree blob
      c. Root filesystem (initramfs)

B. During run time
   1. Linux OS kernel image , decompressed and executing
   2. Root filesystem , mounted as initramfs
   3. Applications Binaries.
   4. Runtime data , process stacks, heaps, page tables
   5. Device tree blob , passed to and used by kernel
   6. Kernel modules , beamformer.ko

C. External SD card

The external SD card has to be preloaded with

A. Boot image
   1. FSBL
   2. PL bitstream
   3. U-boot

B. FIT image
   1. Linux kernel
   2. Device tree blob
   3. Root filesystem

3. Datapath

A. Processing System (PS)

Hosts Linux kernel , runs userspace application, GigE MAC interface, and AXI interface to DDR3 and AXI - 4 LITE interface to PL.

BLUF : boot-up process, you may skip to Section 3.A.e for project specific datapath.

a. bootROOM

HARDWIRED

Upon powering on the device, the bootROM

1. Reads and validates BOOT.BIN from the SD card
2. Copies FSBL (from BOOT.BIN) into OCM
3. Then jumps to FSBL entry point in OCM.

b. FSBL

FROM OCM

1. The FSBL Initialises the PS peripherals including DDR3 memory controller, system clocks, MIO pin configurations, Ethernet , UART , SD interface
2. Programs the PL from BOOT.BIN. After that FSBL loads U-Boot from BOOT.BIN into DDR3
3. Jumps to U-Boot entry point in DDR3.

c. U-Boot

FROM DDR3

The U-Boot is a second stage boot loader it performs the following

1. It initialises remaining peripherals
2. Loads FIT image from SD card into DDR3
3. Unpacks FIT image by decompressing linux kernel to place at DDR3 execution address
4. Places device tree blob to pass to the kernel
5. Extract rootfs to place in DDR3 as initramfs.

d. Linux Kernel Boot

FROM DDR3

1. Decompresses itself (if not already)
2. Reads device tree blob, discovers all PS peripherals and, discovers PL peripherals.
3. Initialises memory management (page tables, virtual memory)
4. Initialises all drivers: DDR3 memory driver, Ethernet driver, UART driver, AXI bus driver, beamformer.ko kernel module is loaded here
5. Mounts root filesystem (initramfs from DDR3)
6. Starts init process (PID 1)
7. Starts system services:
   SSH server (dropbear)
   Network (eth0 via DHCP)
   UART console
8. /dev/beamformer appears which is created by beamformer.ko

e. Linux Userspace

FROM DDR3

1. input_app is loaded from rootfs into DDR3
2. input_app
   input_app is a regular Linux executable binary which is compiled from input.c. Its functionality is to take target angle values from the user via SSH over GigE (Gigabit Ethernet) connection.

```
root@pact-zero:~# ./input_app
Enter azimuth and elevation (e.g. 30,20).
Ctrl+C to stop.
> 30,60
[beamformer] az=30 el=60 → β written to PL
> _
```

3. Linux VFS routes write to
   beamformer.ko (file compiled from beamformer.c)
4. beamformer.ko:
   - Receives target angle values from input
   - Initiates AXI4-Lite transaction
   - Writes these value to AXI-mapped PL registers

Data crosses PS-PL boundary.

B. Programmable Logic (PL)

The basic functionality of the PL in our project is to read the target angle value using the AXI-4 LITE protocol from the PS and to calculate the offset value for each phase shifter IC and send it over through SPI pipeline.

PACT - Zero achieves it thru two deployment configurations.

a. Files common to both configuration

Both the configurations has common implementation for

1. axi_slave.vhd
   AXI-4 LITE slave, to fetch target angle values from the PS and writes into PL registers.

2. spi_driver.vhd
   SPI master takes values from each configuration top level file to write the calculated offset value to phase shifter IC

Offsets for an array element are calculated using the following equations:

$$\beta_x = -k \cdot d_x \cdot \sin(\theta_0) \cdot \cos(\varphi_0) \quad \text{(Eq.1)}$$

$$\beta_y = -k \cdot d_y \cdot \sin(\theta_0) \cdot \sin(\varphi_0) \quad \text{(Eq.2)}$$

[See Section 5.B.1]

Where,

k is the spatial frequency ($k = 2\pi/\lambda$) and,

d_x and d_y represent the physical distance of the element from the X and Y axes, respectively.

For an (m, n) indexed uniform planar array, the element spacing is defined as $d_x = m \cdot \lambda/2$ and $d_y = n \cdot \lambda/2$.

Substituting k ,d_x and d_y into the original equations yields the total phase shift $\beta_{m,n}$ for any element (m, n) in an $M \times N$ planar array:

$$\beta_{m,n} = -\pi \cdot (m \cdot \cos(\varphi_0) \cdot \sin(\theta_0) + n \cdot \sin(\varphi_0) \cdot \sin(\theta_0)) \quad \text{(Eq.3)}$$

Using Eq.3 for our 2x2 UPA,

$$\beta_{0,0} = 0$$
$$\beta_{0,1} = -\pi \cdot \sin(\theta_0) \cdot \sin(\varphi_0)$$
$$\beta_{1,0} = -\pi \cdot \sin(\theta_0) \cdot \cos(\varphi_0)$$
$$\beta_{m,n} = -\pi \cdot (\cos(\varphi_0) \cdot \sin(\theta_0) + \sin(\varphi_0) \cdot \sin(\theta_0))$$

This can be further simplified as using product-to-sum identity

$$\beta_{0,0} = 0$$
$$\beta_{0,1} = -\frac{\pi}{2}\left[\cos(\theta_0 - \varphi_0) - \cos(\theta_0 + \varphi_0)\right]$$
$$\beta_{1,0} = -\frac{\pi}{2}\left[\sin(\theta_0 + \varphi_0) + \sin(\theta_0 - \varphi_0)\right]$$
$$\beta_{1,1} = -\frac{\pi}{2}\left[\sin(\theta_0 + \varphi_0) + \sin(\theta_0 - \varphi_0) + \cos(\theta_0 - \varphi_0) - \cos(\theta_0 + \varphi_0)\right]$$

Taking $\beta_{1,1}$ as reference ($\beta_{1,1}$ = 0)

$$\beta_{0,0} = \frac{\pi}{2}\left[\sin(\theta_0 + \varphi_0) + \sin(\theta_0 - \varphi_0) + \cos(\theta_0 - \varphi_0) - \cos(\theta_0 + \varphi_0)\right] \quad \text{.....(Eq.4)}$$

$$\beta_{0,1} = \frac{\pi}{2}\left[\sin(\theta_0 + \varphi_0) + \sin(\theta_0 - \varphi_0)\right] \quad \text{.....(Eq.5)}$$

$$\beta_{1,0} = \frac{\pi}{2}\left[\cos(\theta_0 - \varphi_0) - \cos(\theta_0 + \varphi_0)\right] \quad \text{.....(Eq.6)}$$

$$\beta_{1,1} = 0 \quad \text{.....(Eq.7)}$$

This representation is chosen as performing addition is less taxing on the hardware than multiplication.

b. Configuration A
PURE HARDWARE LOGIC

1. **phase_compute.vhd**
   Takes target angle from PL registers and calculate offset values using the formulae described in Eq. 4,5,6 and 7 which are derived from Eq.1 and Eq.2.

   Calculates the sin and cos value by instantiating cordic_calc.vhd

2. **cordic_calc.vhd**
   The sin and cos values are calculated by a hardware efficient method called CORDIC [See 5.E.]

3. **pact_zero_top.vhd**
   Top level entity for Configuration A, instantiates all Config A submodules (B.a.1. , B.a.2. , B.b.1. and B.b.3.) and contains the sequencer FSM. The sequencer cycles through all four array elements (send_0/wait_0 × 4 states), feeding phase offsets from phase_compute.vhd to spi_driver.vhd one element at a time. Handles active-low reset conversion. Exposes AXI4-Lite ports for PS connection and SPI ports for phase shifter IC connection.

c. Configuration B
PICORV32 SOFTCORE

4. **picorv32.v**
   PicoRV32 RV32IMC Softcore CPU Unmodified YosysHQ PicoRV32 implementation [See Section 5.A.1. and Section 5.A.2]. Instantiated with COMPRESSED_ISA=1, ENABLE_MUL=1, ENABLE_DIV=1. Exposes standard memory bus: mem_valid, mem_ready, mem_addr, mem_wdata, mem_wstrb, mem_rdata. All unused ports (pcpi, irq, eoi) tied to zero or open.

5. **offset_reg.vhd**
   Memory-Mapped Phase Offset Register File Four 8-bit registers at address nibbles 0x0 (0x20000010), 0x4 (0x20000014), 0x8 (0x20000018), 0xC (0x2000001C). Written by PicoRV32 via mem_intercon , wstrb byte 0 only (8-bit values). Raises ready one clock after valid. Internal signals int_offset_00/10/01/11 driven to output ports. Read by sequencer FSM in picorv32_top for SPI transmission.

6. **mem_intercon.vhd**
   PicoRV32 Memory Bus Interconnect Routes PicoRV32 memory bus transactions to correct peripherals based on address decoding.
   RAM: address bits (31 downto 14) = 0.
   UART: 0x10000000.
   CMD_REG: 0x20000004 , returns cmd_data on mem_rdata.
   Offset registers: address bits (31 downto 4) = 0x2000001
   Drives mem_ready from appropriate peripheral ready signal.
   mem_rdata mux: ram_rdata / cmd_data / zeros.
   Uses buffer ports for valid signals so they can be read internally.

7. **picorv32_top.vhd**
   Top Level Entity for Configuration B Instantiates all Config B submodules (5.a.1 , 5.a.2 , 5.c.4 , 5.c.5 , 5.c.6) and simulation stub (5.c.8 and 5.c.9). Contains sequencer FSM cycles through four elements feeding offset_reg.vhd outputs to spi_driver.vhd. Exposes same AXI4-Lite and SPI ports as for drop-in compatibility. Internal signals: int_cmd_data, int_cmd_write connects axi_slave.vhd to mem_intercon; int_offset_valid, int_offset_ready connect mem_intercon to offset_reg.

8. **ram_simulation.vhd**
   This is a BRAM simulation. Implements a 4096-word (16KB) synchronous RAM initialised at elaboration time from firmware.hex using VHDL readmemh(). Responds to mem_valid with one-cycle read/write latency. Supports byte-level writes via mem_wstrb each strobe bit enables writing the corresponding byte of mem_wdata. Address decoded from mem_addr(13 downto 2) , word-aligned 32-bit access. Maps to PicoRV32 address range 0x00000000–0x00003FFF. Not synthesisable, simulation only.
   In synthesis this would be replaced by Xilinx BRAM IP with firmware pre-initialised in the bitstream.

9. **uart_controller.vhd**
   Accepts write transactions from PicoRV32 at address 0x10000000. Each write is treated as a single ASCII character extracted from wdata(7 downto 0) and printed to the Vivado XSim Tcl console via VHDL write() and writeline(). Raises ready immediately on valid. Enables firmware print_str() calls to produce visible output during simulation without requiring a real UART peripheral.
   Not synthesisable , simulation only.
   In synthesis this would be replaced by a Xilinx UART Lite IP or PS UART connected via AXI.

10. **main.c**
    PicoRV32 Bare Metal Firmware Runs on PicoRV32 softcore. Contains: 91-entry sin lookup table (0–90°, scaled by 256); sin_fixed(int) function with quadrant handling (no division, no multiplication); cos_fixed(int) implemented as sin_fixed(90-angle); compute_offsets() computing delta_x and delta_y using product-to-sum identity; main() polling loop reading CMD_REG at 0x20000004, detecting value change, calling compute_offsets(), writing results to four offset registers at 0x20000010–0x2000001C.

11. **startup.S**
    RISC-V Bare Metal Startup Placed in .text.start section to guarantee first execution. Sets stack pointer to 0x00004000 (top of 16KB RAM). Jumps to main(). Infinite loop after main returns.

12. **linker.ld**
    Linker Script Defines single RAM region at 0x00000000, 16KB. Section order: .text.start first (startup code), then .text (code), .rodata (sin_lookup table and string literals), .data, .bss.

C. Register Map

Location	Register Name	Size	Access	Description	Used in
0x00	Steering Azimuth	32 bit	PS write, PL read	target azimuth here	Configuration A
0x04	Steering Elevation	32 bit	PS write, PL read	target elevation here	Configuration A
0x08	PHASE_REG_0	32 bit		Reserved	
0x0C	PHASE_REG_1	32 bit		Reserved	
0x10	PHASE_REG_2	32 bit		Reserved	
0x14	PHASE_REG_3	32 bit		Reserved	
0x18	CONTROL_REG	32 bit		Reserved	
0x1C	STATUS_REG	32 bit		Reserved	
0x20	CMD_REG	32 bit	PS write, PicoRV32 read	target azimuth and elevation	Configuration B

Table 1 : FPGA Register Map

4. File Structure

	File Type	File Name	File location
PACT Zero	Synthesisable VHDL	axi_slave.vhd	/rtl
		spi_driver.vhd	/rtl
		pact_zero_top.vhd	/rtl
		phase_compute.vhd	/rtl
		cordic_calc.vhd	/rtl
		picorv32_top.vhd	/rtl
		mem_intercon.vhd	/rtl
		offset_reg.vhd	/rtl
	Simulation-Only	ram_simulation.vhd	/rtl
		uart_controller.vhd	/rtl
	Softcore	startup.S	/softcore/fw
		linker.ld	/softcore/fw
		main.c	/softcore/fw
		firmware.elf	compiled from startup.S + linker.ld + main.c via: riscv64-unknown-elf-gcc
		firmware.hex	compiled from firmware.bin via: python3 makehex.py makehex.py from YosysHQ
		picorv32.v	/softcore/core from YosysHQ
	PS driver	beamformer.c	/driver
		beamformer.bb	PetaLinux Yocto Receipe
		Makefile	Kernel module build script from PetaLinux
		input.c	/driver
		beamformer.ko	compiled from beamformer.c via: make (kernel build system)
		input_app	compiled from input.c via: arm-linux-gnueabihf-gcc

5. References

A. Open Source Attributions

1. YosysHQ PicoRV32

PicoRV32 is a CPU core that implements the RISC-V RV32IMC Instruction Set. It can be configured as RV32E, RV32I, RV32IC, RV32IM, or RV32IMC core, and optionally contains a built-in interrupt controller.

2. RISC-V RV32IMC ISA

RISC-V RV32IMC ISA specification, used as the instruction set reference for PicoRV32 integration.

https://docs.riscv.org/reference/home/index.html

B. Equations

Equation 1 and 2

1. Balanis, C. A., *Antenna Theory: Analysis and Design*, 4th ed., John Wiley & Sons, 2016, Chapter 6: Arrays (Linear, Planar, and Circular).

C. Zynq 7000 series

1. AMD Xilinx Zynq-7000 All Programmable SoC Product Page https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html

D. Arm® AMBA® AXI-4 LITE

1. https://developer.arm.com/documentation/ihi0022/e

E. CORDIC

This algorithm is selected for Configuration A to calculate the sine and cosine trigonometric functions required by equations (Eq. 4,5,6 and 7) without requiring dedicated hardware multipliers.

1. J. E. Volder, "The CORDIC Trigonometric Computing Technique," IRE Transactions on Electronic Computers, vol. EC-8, no. 3, pp. 330-334, Sept. 1959, doi: 10.1109/TEC.1959.5222693.

6. Supplemental Documents

A. Errata

All the issues arising from the PACT-Zero project is documented at https://github.com/sonixaman/PACT/tree/main/PACT-Zero/Errata

For reporting functional issues, see Section 7

B. Simulation Result

All the simulation results of the PACT-Zero project is documented at https://github.com/sonixaman/PACT/tree/main/PACT-Zero/sim

7. Glossary

A. AXI4-Lite
Advanced eXtensible Interface 4 Lite. A simplified subset of the ARM AMBA AXI4 protocol used for low-bandwidth register-style communication between PS and PL on Zynq SoCs. Used in PACT Zero for PS to PL angle value transfer.

B. Bare Metal
Software running directly on hardware without an operating system. In PACT Zero, main.c runs bare metal on the PicoRV32 softcore.

C. Beamforming
The process of controlling the phase and/or amplitude of signals fed to individual antenna elements in a phased array to steer the radiated beam in a desired direction.

D. BRAM
Block RAM. Dedicated on-chip memory blocks in Xilinx FPGA fabric. In PACT Zero simulation, ram_simulation.vhd models BRAM behaviour. In real hardware, BRAM would be initialised with firmware.hex at bitstream generation.

E. Bitstream
Binary file (.bit) generated by Vivado that configures the FPGA programmable logic fabric. Contains the hardware implementation of all RTL source files. Packaged inside BOOT.BIN and loaded into PL by FSBL during boot.

F. BOOT.BIN
Zynq Boot Image. A single binary file generated by Xilinx Bootgen containing FSBL, PL bitstream, and U-Boot. Stored on SD card and read by BootROM at power-on.

G. BootROM
Read-Only Memory hardwired into Zynq silicon. Cannot be modified. First code executed after power-on. Reads BOOT.BIN from SD card, copies FSBL into OCM, and jumps to FSBL entry point.

H. CMD_REG
Command Register. AXI register at offset 0x20 in axi_beamformer_slave. Used exclusively in Configuration B. PS writes packed azimuth and elevation (elevation[31:16] | azimuth[15:0]) in a single 32-bit write. PicoRV32 reads this at memory address 0x20000004.

I. CORDIC
COordinate Rotation DIgital Computer. An iterative hardware algorithm for computing trigonometric functions (sin, cos) using only addition, subtraction, and bit shifts, no hardware multipliers required. Used in Configuration A (cordic_calc.vhd) with a 16-stage pipeline.

J. DDR3 SDRAM
Double Data Rate 3 Synchronous Dynamic RAM. External memory connected to the Zynq PS memory controller. Provides 1GB working memory for the ARM processor. Holds Linux kernel, root filesystem, kernel modules, and application binaries at runtime.

K. Device Tree Blob (DTB)
Binary representation of the hardware description passed to the Linux kernel at boot. Describes all PS peripherals and PL peripherals (including AXI slave at 0x40000000) so the kernel can initialise the correct drivers.

L. ELF
Executable and Linkable Format. Standard binary format for compiled executables and object files on Linux and embedded systems. firmware.elf is the compiled PicoRV32 firmware before binary extraction.

M. Elevation (θ)
The vertical steering angle of the phased array beam measured from the horizontal plane. One of two angles (with azimuth) used to specify beam direction.

N. Azimuth (φ)
The horizontal steering angle of the phased array beam measured in the horizontal plane. One of two angles (with elevation) used to specify beam direction.

O. FIT Image
Flattened Image Tree. A U-Boot image format that packages multiple binary components (Linux kernel, device tree blob, root filesystem) into a single file (image.ub). Loaded by U-Boot from SD card into DDR3 during boot.

P. FPGA
Field-Programmable Gate Array. Reconfigurable hardware fabric consisting of programmable logic cells, DSP blocks, BRAM, and routing. In Zynq-7000, the PL (Programmable Logic) is the FPGA portion of the SoC.

Q. FSM
Finite State Machine. A sequential logic circuit that transitions between a finite set of states based on inputs. Used in PACT Zero as the sequencer FSM in both configurations to cycle through the four array elements for SPI transmission.

R. FSBL
First Stage Bootloader. Runs from OCM immediately after BootROM. Initialises DDR3 memory controller, system clocks, MIO pins, and PS peripherals. Programs PL with bitstream. Loads U-Boot into DDR3.

S. GigE MAC
Gigabit Ethernet Media Access Controller. Hard IP peripheral built into the Zynq PS silicon. Provides 1Gbps Ethernet connectivity. Used in PACT Zero for SSH access to the Linux shell over Ethernet no custom RTL required.

T. HDL
Hardware Description Language. A language used to describe digital hardware behaviour and structure. VHDL and Verilog are the two most common HDLs. PACT Zero uses VHDL for all custom RTL.

U. image.ub
The FIT image file stored on the SD card. Contains Linux kernel, device tree blob, and root filesystem packaged together. Loaded by U-Boot during boot.

V. initramfs
Initial RAM Filesystem. A root filesystem loaded entirely into DDR3 RAM at boot. Used by default in PetaLinux builds. The entire rootfs including beamformer.ko and input_app lives in DDR3 at runtime.

W. iowrite32
Linux kernel function for writing a 32-bit value to a memory-mapped I/O address. Used in beamformer.c to write steering angles to AXI-mapped PL registers.

X. ioremap
Linux kernel function that maps a physical address range (e.g. AXI base address 0x40000000) into kernel virtual memory, making it accessible via pointer dereference or iowrite32/ioread32.

Y. JTAG
Joint Test Action Group. A hardware debugging interface. Used for FPGA programming and debugging. On ZC702, JTAG is accessible via the Digilent USB-JTAG adapter.

Z. Kernel Module (.ko)
A loadable Linux kernel module. Runs in kernel space with direct hardware access. beamformer.ko is compiled from beamformer.c and loaded automatically at Linux boot, registering /dev/beamformer.

AA. Linker Script (.ld)
A file that instructs the linker how to arrange code and data sections in memory. linker.ld in PACT Zero defines a single 16KB RAM region at 0x00000000 and specifies section placement order for the PicoRV32 firmware.

BB. mem_intercon
Memory Interconnect. VHDL module in Configuration B that routes PicoRV32 memory bus transactions to the correct peripheral based on address decoding. Acts as the address decoder and bus arbiter for the PicoRV32 memory space.

CC. MIO
Multiplexed I/O. A bank of 54 configurable pins on the Zynq PS that can be assigned to different PS peripherals (UART, Ethernet, SD card, SPI, I2C etc.). Configured by FSBL and Vivado PS settings.

DD. MOSI
Master Out Slave In. The SPI data line on which the master (spi_driver.vhd) transmits data to the slave (phase shifter IC).

EE. OCM
On-Chip Memory. 256KB of SRAM built into the Zynq PS silicon. Used by FSBL during early boot before DDR3 is initialised. Too small for Linux all runtime execution occurs in DDR3.

FF. offset_reg
Phase Offset Register File. VHDL module in Configuration B. Four 8-bit memory-mapped registers (0x20000010–0x2000001C) written by PicoRV32 and read by the sequencer FSM for SPI transmission.

GG. PetaLinux
Xilinx's embedded Linux development tool. Builds the complete Linux software stack for Zynq including FSBL, U-Boot, Linux kernel, device tree, and root filesystem. Generates BOOT.BIN and image.ub.

HH. PicoRV32
A small RISC-V CPU core implementing the RV32IMC instruction set, developed by YosysHQ (Claire Wolf). Used in PACT Zero Configuration B as a softcore CPU instantiated in the PL fabric.

II. PL
Programmable Logic. The FPGA fabric portion of the Zynq-7000 SoC. Implements hardware logic described by RTL source files. In PACT Zero, PL implements the AXI slave, beamforming pipeline, and SPI master.

JJ. PS
Processing System. The fixed ARM Cortex-A9 processor portion of the Zynq-7000 SoC. Runs Linux OS, kernel drivers, and userspace applications. Communicates with PL via AXI4-Lite bus.

KK. Product-to-Sum Identity
Trigonometric identity: $\sin(A+B) + \sin(A-B) = 2\cdot\sin(A)\cdot\cos(B)$ and $\cos(A-B) - \cos(A+B) = 2\cdot\sin(A)\cdot\sin(B)$. Used in PACT Zero to compute phase offsets using only addition of sin/cos values rather than multiplication more hardware efficient.

LL. readmemh
VHDL system task that reads a hex file and initialises a memory array at simulation elaboration time. Used in ram_simulation.vhd to load firmware.hex into the simulated BRAM.

MM. RISC-V
Reduced Instruction Set Computer - Five. An open standard instruction set architecture (ISA). RV32IMC denotes 32-bit base integer (I), multiply/divide extension (M), and compressed 16-bit instructions (C).

NN. rootfs
Root Filesystem. The top-level filesystem containing all Linux system files,

libraries, kernel modules, and application binaries. In PACT Zero, rootfs is packaged as initramfs inside image.ub and loaded into DDR3 at boot.

OO.RTL
Register Transfer Level. A design abstraction describing digital hardware in terms of data flow between registers and the logic operations performed on that data. VHDL files in PACT Zero are RTL source files.

PP. SCK
Serial Clock. The SPI clock signal generated by the SPI master (spi_driver.vhd) to synchronise data transmission to the phase shifter ICs.

QQ.SDIO
Secure Digital Input/Output. The hardware interface on the Zynq PS used to communicate with the SD card. Connected via MIO pins 40–45 on ZC702.

RR.Softcore CPU
A CPU implemented in programmable logic (FPGA fabric) as RTL, as opposed to a hardened silicon CPU. PicoRV32 is a softcore CPU instantiated in the Zynq PL in Configuration B.

SS. SoC
System on Chip. A single integrated circuit containing multiple components processor, memory controller, peripherals, and programmable logic. The Zynq-7000 XC7Z020 is a SoC combining ARM Cortex-A9 PS and FPGA PL.

TT. SPI
Serial Peripheral Interface. A synchronous serial communication protocol using four signals: SCK (clock), MOSI (master to slave data), MISO (slave to master data), and CS_N (active-low chip select). Used in PACT Zero to send 6-bit phase commands to phase shifter ICs.

UU.U-Boot
Universal Bootloader. Second stage bootloader running from DDR3. Loads the FIT image from SD card, unpacks kernel, device tree, and rootfs into DDR3, then jumps to the Linux kernel entry point.

VV. UPA
Uniform Planar Array. A 2D antenna array with elements arranged in a regular grid with equal spacing. PACT Zero implements a 2×2 UPA.

WW. VHDL
: Very High Speed Integrated Circuit Hardware Description Language. HDL used for all RTL source files in PACT Zero.

XX. Vivado
: Xilinx/AMD FPGA design suite. Used in PACT Zero for RTL synthesis, implementation, bitstream generation, and behavioural simulation (XSim).

YY. wstrb
: Write Strobe. A 4-bit signal in AXI4-Lite and PicoRV32 memory bus indicating which bytes of the 32-bit write data are valid. Bit 0 = byte 0 (bits 7:0), bit 1 = byte 1 (bits 15:8), etc.

ZZ. XSim
: Xilinx Simulator. Built-in behavioural simulator in Vivado. Used to verify PACT Zero RTL designs before synthesis. Configuration A SPI TEST PASSED in XSim.

AAA. Yocto
: An open-source build system for embedded Linux. PetaLinux is built on top of Yocto. beamformer.bb is a Yocto recipe that packages beamformer.c into the PetaLinux rootfs.

BBB. ZC702
: Xilinx Zynq-7000 All Programmable SoC evaluation board. Features XC7Z020 SoC, 1GB DDR3 SDRAM, SD card slot, GigE Ethernet, HDMI, USB, and JTAG. Target hardware platform for PACT Zero.

8. Contact Info

Aman Soni
B.Tech Graduate
Electrical & Electronics Engineering
Manipal Institute of Technology, Manipal

GitHub : github.com/sonixaman
Email : post.amansoni@gmail.com
LinkedIn : linkedin.com/in/amansoni25/

———End of Document———